🛠️ Ejercicios

Tabla de contenidos

1 Trasfromando funciones 1
2 Fábrica de promociones 3
3 Bendita media 3
4 Sucesión de Fibonacci 4
5 Palíndromos recursivos 4
  5.1 Punto extra 5
6 Área de aprendizaje 5
7 Socios ordenados 5
  7.1 Punto extra 6
8 Listado de rimas 6
9 Analistas de temperaturas 6
  9.1 Enfoque funcional 7
  9.2 Enfoque idiomático 7
10 El tiempo vuela 7
  10.1 Punto extra 7
11 No perdamos el centro 8
12 En Python es mejor 8
13 Subiendo de rango 9
14 La cajita musical 9
15 El mejor precio 10
16 Bromas pesadas 😱 11
17 *Pipelines* de procesamiento 😱 12
18 ¿Espacio o tiempo? ⏳ 13

1 Trasfromando funciones

Debajo se muestra un listado de funciones impuras. Analice por qué son impuras e implemente alternativas puras (manteniendo su lógica):

Función 1

```
contador = 0
def incrementar():
    global contador
    contador += 1
    return contador
```

Línea 3

La palabra `global` es una *keyword* que indica que se pretende usar y **modificar** una variable definida fuera del ámbito de ejecución de la función.

Función 2

```
def obtener_hora_actual():
    import datetime
    return datetime.datetime.now().hour
```

Función 3

```
def add_time(time, hours, minutes, seconds):
    increment_time(time, hours, minutes, seconds)
    return time
```

Línea 2

Asuma que esta función incrementa a `time`, que es un objeto `datetime`, en `hours` horas, `minutes` minutos y `seconds` segundos.

Ayuda

Represente fechas y horas con el objeto objeto `datetime` del módulo estándar `datetime`. Además, considere el objeto `timedelta` del mismo módulo para luego calcular diferencias. Por ejemplo:

```
import datetime

ahora = datetime.datetime.now() # Devuelve un objeto con la fecha y la hora
actual
print(ahora)

una_hora_mas_tarde = datetime.timedelta(hours=1, minutes=0, seconds=0)
print(ahora + una_hora_mas_tarde)
```

```
2025-08-31 09:48:58.327461
2025-08-31 10:48:58.327461
```

Función 4

```
historial_de_nombres = []
def registrar_nombre(nombre):
    historial_de_nombres.append(nombre)
    return f"'{nombre}' ha sido registrado en el historial."
```

Función 5

```
LIMITE_MAXIMO = 100
def verificar_limite(valor):
    if valor > LIMITE_MAXIMO:
        return "Excede el límite"
    return "Dentro del límite"
```

2 Fábrica de promociones

Considere el ejercicio Promociones de la Práctica 1. Utilizando el mismo sistema de descuentos cree la función `crear_promocion` que reciba como argumento el medio de pago y devuelva una función que al aplicarse sobre un número aplique el descuento (o recargo) que le corresponde a ese medio de pago. Por ejemplo:

```
promo_debito = crear_promocion("débito")
print(promo_debito(1000))
print(promo_debito(2700))
```

```
900
2430.0
```

Luego, cree la función `crear_promocion_personalizada` que recibe el medio de pago y, de manera opcional, el porcentaje de descuento a aplicar. Como resultado devuelve una función que al aplicarse sobre un número impacta el descuento o recargo correspondiente. Además, considere que:

- Si no se pasa el porcentaje a aplicar, se deben usar los descuentos y recargos detallados en el enunciado del ejercicio en la Práctica 1.
- Caso contrario, la función devuelta debe aplicar ese porcentaje personalizado (e.g., `crear_promocion_personalizada("débito", 15)` para 15% de descuento).

Ejemplo de uso:

```
promo_debito = crear_promocion_personalizada("débito", 15)
promo_debito(1000)
```

```
850
```

3 Bendita media

En Python no existe una función *built-in* que calcule la media de una secuencia de números. El objetivo de este ejercicio es implementar una función `mean` que funcione tanto cuando se le pasa un iterable, como cuando se le pasa una cantidad arbitraria de números.

El argumento es un iterable:

```
mean([6.27, 8.11, 7.6, 5.2, 4.8])
```

Se pasan una cantidad arbitraria de valores numéricos:

```
mean(7.3, 8.2, 11.0, 12.5)
```

4 Sucesión de Fibonacci

Considere la sucesión que comienza por los números 0 y 1. Los siguientes números se forman sumando los dos anteriores.

$$\{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \cdots\}$$

Esta sucesión se conoce como **sucesión de Fibonacci**.

Implemente una función recursiva que tome un número natural `n` como entrada y devuelva el n--ésimo número en la sucesión.

5 Palíndromos recursivos

Un palíndromo es una palabra, frase, número o secuencia que se lee igual de izquierda a derecha que de derecha a izquierda. En Argentina, a un número de este tipo le decimos le decimos capicúa.

Para comprobar en Python si una cadena es palíndroma, puede compararse con su versión invertida, que se construye mediante un slice con paso −1:

```
c = "anilina"
c == c[::-1]
```

```
True
```

También podríaimos determinar si una cadena es un palíndromo de manera recursiva comparando el primer y el último carácter:

- Si la cadena tiene longitud 0 o 1, se devuelve `True`.
- Si el primer y el último carácter son iguales, se llama recursivamente a la función pasándole la cadena excluyendo al primer y último caracter ya comparados. Si son distintos, se devuelve `False`.

En otras palabras, la función recursiva devolverá `True` cuando todas las comparaciones por pares resulten verdaderas y se alcance el caso base: queda un único carácter (longitud impar) o ninguno (longitud par).

Ayuda

Para seleccionar todos los caracteres de una cadena, exceptuando al primero y el último, se puede usar

```
"radar"[1:-1]
```

```
"ada"
```

5.1 Punto extra

Adapte la función para que no considere espacios ni distinga mayúsculas de minúsculas. De este modo, debería detectar que la siguiente frase es palíndroma.

```
palindromo("Anita lava la tina") # True
```

6 Área de aprendizaje

Se cuenta con una lista de tuplas de longitud 2, representando el ancho y alto de distintos rectángulos.

```
rectangulos = [
    (5, 8),
    (2, 2),
    (9, 2),
    (3, 3),
    (3, 7),
    (6, 3)
]
```

Cree una nueva lista que ordene dichos rectángulos en función de su área utilizando la función `sorted`.

7 Socios ordenados

Se cuenta con la siguiente lista de diccionarios, la cual contiene datos personales sobre miembros de un club de atletismo:

```
datos_socios = [
    {"nombre": "Bautista Carrara", "edad": 22, "altura_cm": 178,
"record_100m": 13.4},
    {"nombre": "Valentina Lucci",  "edad": 23, "altura_cm": 163,
"record_100m": 14.2},
    {"nombre": "Gerónimo Cuesta",  "edad": 26, "altura_cm": 170,
```

```
"record_100m": 14.0},
    {"nombre": "Lucio Borga",      "edad": 28, "altura_cm": 186,
"record_100m": 13.8},
    {"nombre": "Julia Spoglia",    "edad": 21, "altura_cm": 163,
"record_100m": 11.9},
    {"nombre": "Soledad Colombo",  "edad": 22, "altura_cm": 170,
"record_100m": 13.5}
]
```

Ordene la lista en base a los récords en la carrera de 100 metros, en forma ascendente.

7.1 Punto extra

Implemente una función que tome como argumento una clave de diccionario y devuelva una lista ordenada por los valores de dicha clave. Si el argumento toma el valor `"nombre"`, ordene los elementos alfabéticamente en base a los apellidos.

8 Listado de rimas

Se tiene la siguiente lista de palabras, la cual se quiere utilizar para formar rimas:

```
palabras_a_rimar = [
    "actividad",
    "bendición",
    "cartelera",
    "ciudad",
    "escalera",
    "estación",
    "felicidad",
    "función",
    "reposera"
]
```

Ordene la lista en base al orden alfabético del reverso de cada palabra, de modo que las palabras que riman se encuentren juntas.

> Ayuda
>
> Si la lista fuese `["durazno", "kiwi"]`, el resultado sería `["kiwi", "durazno"]`, porque `"iwik"` precede a `"onzarud"` en orden alfabético.

9 Analistas de temperaturas

Considere una lista de temperaturas en grados Celsius sobre la cual se deben aplicar distintas operaciones.

```
temperaturas_celsius = [
    25.5, 28.0, 19.3, 31.5, 22.8, 17.0, 30.2, 35.6, 14.2,
    32.4, 22.7, 10.1, 29.5, 33.9, 22.1, 38.9, 18.4, 16.3
]
```

9.1 Enfoque funcional

1. Convierta las temperaturas a grados Fahrenheit utilizando `map` y almacene el resultado en una lista llamada `temperaturas_f`. Use la fórmula de conversión:

$$F = C \times \frac{9}{5} + 32$$

2. Utilice `filter` para seleccionar de la lista anterior las temperaturas que sean mayores a 80°F. Guarde el resultado en una nueva lista. ¿Se le ocurre una alternativa que no utilice la lista creada en el primer punto?

9.2 Enfoque idiomático

1. Utilice una *list comprehension* para obtener una lista de temperaturas en grados Fahrenheit solo cuando para temperaturas mayores a 22 °C.

10 El tiempo vuela

Se quiere medir el tiempo que tarda la computadora en ejecutar distintos bloques de código. Para eso, implemente una función `crear_cronometro` que fabrique una función `cronometro`, la cual devuelve el tiempo transcurrido entre su creación y la llamada a la función. Luego, utilice dos cronómetros en paralelo para evaluar el siguiente código:

```
cronometro1 = crear_cronometro()

for i in range(10**4):
    i ** 2 # Calcula el cuadrado de un número pero no lo devuelve

print(f"El bloque entero tardó {cronometro1()} segundos en ejecutarse.")

cronometro2 = crear_cronometro()

for j in range(10**6):
    j // 2 # Calcula la división entera por 2 pero no la devuelve

print(f"El segundo bucle tardó {cronometro2()} segundos en ejecutarse.")
```

10.1 Punto extra

Modifique el funcionamiento del cronómetro para que en cada llamada devuelva el tiempo transcurrido entre la llamada actual y la inmediata anterior (excepto en la primera llamada, que devuelve el tiempo transcurrido desde su creación).

Ayuda

Para medir el paso del tiempo en Python podemos usar la función `time` del módulo homónimo.

```
from time import time, sleep

inicio = time()
sleep(2) # Detiene la ejecución por 2 segundos
print(time() - inicio) # ~ 2 (segundos)
```

11 No perdamos el centro

Suponga el siguiente listado de números, que contiene algunos `None`:

```
numeros = [
    2.05, 1.09, None, 2.31, 2.28, 0.97, 2.59, 2.72, 0.76, None, 1.88, 2.04,
3.25, 1.88, None
]
```

Calcule la media de los numeros, sin considerar los nulos. Luego, utilizando una *list comprehension*, obtenga una nueva lista de los valores centrados, conservando los `None` en las posiciones originales.

12 En Python es mejor

Considere el siguiente programa, que selecciona valores atípicos de un listado:

```
numeros = [
    4.74346239e-01, -2.90877176e-01, -1.44377789e+00, -4.48680759e+01,
    -1.21249801e+00, -3.32729317e-01,  2.21676912e-01,  1.05599711e+00,
    -3.62372053e+00, -2.96441579e-01, -4.28304222e+00,  1.55908820e+02,
    9.00858234e-01, -1.09384173e+00, -1.51083571e+00, -5.38491167e-01,
    -3.84153084e-02,  1.20393395e+00,  1.82651406e-01,  2.05179405e+00
]

def media(x):
    return sum(x) / len(x)

def varianza(x):
    numerador = 0
    x_media = media(x)
    for x_i in x:
        numerador += (x_i - x_media) ** 2
    return numerador / len(x)

x_media = media(numeros)
```

```
x_desvio = varianza(numeros) ** 0.5

map_obj = map(lambda x: (x - x_media) / x_desvio, numeros)
list(filter(lambda x: abs(x) > 3, map_obj))
```

Implemente un programa equivalente haciendo uso de una *list comprehension*, en vez de `map` y `filter`.

13 Subiendo de rango

La función `range(start, stop, step)` de Python devuelve un objeto que genera una secuencia de números desde `start` (inclusive) hasta `stop` (exclusive) en incrementos de `step` unidades. El argumento `step`, sin embargo, sólo puede ser un número entero (excepto cero). Implemente una función llamada `frange` que acepte los mismos argumentos, pudiendo `step` ser de tipo `float`. La función debe retornar un **generador** de la secuencia correspondiente.

Ayuda

Utilice el siguiente ejemplo a modo de control:

```
for i in frange(3, 4, 0.2):
    print(f"{i:.2f}")
```

```
3.00
3.20
3.40
3.60
3.80
```

Tenga en cuenta que la precisión finita de las computadoras puede afectar el comportamiento de su generador.

14 La cajita musical

Tenemos una caja musical que recita los siguientes versos:

```
versos = [
    "Tengo que confesar que a veces no me gusta tu forma de ser",
    "Luego te me desapareces y no entiendo muy bien por qué",

    "No dices nada romántico cuando llega el atardecer",
    "Te pones de un humor extraño con cada luna llena al mes",

    "Pero a todo lo demás le gana lo bueno que me das",
```

```
    "Sólo tenerte cerca, siento que vuelvo a empezar"
]
```

Implemente una función para darle cuerda a la caja musical. En cada llamada debe devolver un verso distinto, hasta agotarlos todos.

> Ayuda
>
> Mediante el uso de `yield` se puede lograr que una función se detenga en un punto intermedio y retome desde ese punto en la siguiente llamada.
>
> ```
> def mostrar_fase():
>     print("Inicio")
>     yield
>     print("Medio")
>     yield
>     print("Desenlace")
>     yield
> ```

15 El mejor precio

Un supermercado ofrece múltiples promociones:

- 15% de descuento los días lunes y miércoles.
- 10% de descuento en compras con monto superior a $50.000.
- 20% de descuento a clientes mayores de 65 años.

Para determinar la promoción a aplicar, el supermercado utiliza un programa con la siguiente estructura:

```
def promo_dia_semana(compra):
    """Aplica un 15% de descuento si la compra se realiza un lunes o
miércoles."""
    return None # hay que implementar esta función

def promo_monto_grande(compra):
    """Aplica un 10% de descuento si la compra tiene un monto superior a
$50.000."""
    return None # hay que implementar esta función

def promo_edad(compra):
    """Aplica un 20% de descuento si el cliente tiene 65 años o más."""
    return None # hay que implementar esta función

promos = [promo_dia_semana, promo_monto_grande, promo_edad]
```

```python
def mejor_promo(compra):
    """Construye diccionario con el monto luego de aplicar la mejor promoción.

    Esta función devuelve un diccionario con el monto original, el monto
final, y el descuento
    aplicado.
    """
    # Obtener el multiplicador del mayor descuento
    multiplicador = sorted([promo(compra) for promo in promos])[0]

    return {
        "monto_original": compra["monto"],
        "monto_final": compra["monto"] * multiplicador,
        "descuento": f"{round((1 - multiplicador) * 100)}%"
    }

ejemplo_compra = {"dia": "miércoles", "edad_cliente": 42, "monto": 66420}
mejor_promo(ejemplo_compra)
# {'monto_original': 66420, 'monto_final': 56457.0, 'descuento': '15%'}
```

El problema con esta implementación es que, cada vez que se añade o elimina una promoción, el cambio debe llevarse a cabo tanto en la función de la promoción como en la lista de promociones. Para evitar el trabajo duplicado, implemente los siguientes cambios:

1. Defina primero la lista `promos`, la cual comienza estando vacía.
2. Defina un decorador `promo` que añade una función a la lista `promos` antes de ejecutarla.
3. Implemente las tres funciones de promoción y decórelas con el decorador del paso anterior.

16 Bromas pesadas 😱

Nuestro amigo programador está armando una página web, con una función que saluda a los nuevos usuarios por su nombre cuando se registran. Nosotros queremos gastarle una broma a nuestro amigo, metiendo en su código un decorador que haga que su función corra normalmente excepto cada n-ésima corrida, fallando silenciosamente (no imprime nada). El valor `n` es un número entero de nuestra elección.

```python
@romper_cada(3)
def saludar(nombre):
  print(f"¡Hola, {nombre}!")

saludar("Carlos")     # "¡Hola, Carlos!"
saludar("María Luz")  # "¡Hola, María Luz!"
saludar("Mirna")      # Nada (la función devuelve None)
saludar("Diego")      # "¡Hola, Diego!"
```

Ayuda

Para crear un decorador que reciba argumentos, podemos crear una fábrica de decoradores:

```
def mi_decorador(n):
    def decorar(funcion):
        print(f"Ejecutando decorador con argumento {n}")
        return funcion
    return decorar

@mi_decorador(7)
def imprimir(mensaje):
    print(mensaje)

imprimir("Hola mundo")
# > Ejecutando decorador con argumento 7
# > Hola mundo
```

17 *Pipelines* de procesamiento 😱

Este ejercicio tiene como objetivo implementar un sistema de preprocesamiento para una lista de diccionarios, donde cada diccionario representa una fila con sus columnas como pares clave–valor. Un conjunto de datos de ejemplo es el siguiente:

```
datos = [
    {"edad": 20, "ingresos": 2000},
    {"edad": 30, "ingresos": 3000},
    {"edad": None, "ingresos": 2500},
    {"edad": 40, "ingresos": None},
    {"edad": 25, "ingresos": 4000},
]
```

El primer paso consiste en definir tres funciones:

1. `eliminar_nulos`: elimina las filas con al menos un valor `None`.
2. `calcular_log`: que calcula el logaritmo en base 10 para los valores de la variable indicada.
3. `filtrar`: recibe el listado, el nombre de una columna y una función booleana que se aplica para determinar que registros se conservan.

Que deben funcionar como se muestra en los ejemplos:

```
eliminar_nulos(datos)
```

```
[
    {"edad": 20, "ingresos": 2000},
```

```
    {"edad": 30, "ingresos": 3000},
    {"edad": 25, "ingresos": 4000},
]
```

```
calcular_log(datos, "ingresos")
```

```
[
    {"edad": 20, "ingresos": 3.301},
    {"edad": 30, "ingresos": 3.477},
    {"edad": None, "ingresos": 3.397},
    {"edad": 40, "ingresos": None},
    {"edad": 25, "ingresos": 3.602},
]
```

```
filtrar(datos, "edad", lambda e: e > 25)
```

```
[
    {"edad": 30, "ingresos": 3000},
    {"edad": 40, "ingresos": None},
]
```

En el segundo paso, se debe implementar una función `crear_pipeline`, que recibe una **cantidad arbitraria** de funciones de procesamiento, junto a sus argumentos, y debe devolver una función que, al pasarle un listado de datos, los aplica de manera secuencial y devuelve un conjunto de datos procesado. Por ejemplo:

```
pipeline = crear_pipeline(
    {"fun": eliminar_nulos, "kwargs": {}},
    {"fun": calcular_log, "kwargs": {"var_name": "ingresos"}},
    {"fun": filtrar, "kwargs": {"var_name": edad, "key": lambda e: e > 25}}
)
pipeline(datos)
```

```
[{"edad": 30, "ingresos": 3.477}]
```

18 ¿Espacio o tiempo? ⏳

Suponga que trabaja en el equipo de análisis de datos de un *e-commerce* y suponga que recibe un listado ventas del último mes. Deberá extraer información clave de esos datos utilizando dos maneras distintas de resolver cada tarea.

1. Preparación de los datos

Para empezar, use la siguiente función que genera datos de ejemplo de ventas de distinto tamaño:

```
import random

CATEGORIAS = ("electrónica", "hogar", "accesorios", "deportes")

def generar_ventas(n=100_000, seed=None):
    """Generar datos de ventas.

    Parameters
    ----------
    n : int
        Cantidad de ventas a simular.
    seed : int, optional
        Semilla para el generador de números aleatorios. Por defecto, `None`.

    Returns
    -------
    list[dict]
        Listado de ventas.
        Cada venta es un diccionario con claves `"id"`, `"precio"` y
`"categoria"`.
    """
    rng = random.Random(seed)
    ventas = []
    for i in range(1, n + 1):
        ventas.append(
            {
                "id": f"P{i+1:06d}",
                "precio": round(rng.uniform(5.0, 500.0), 2),
                "categoria": CATEGORIAS[i % len(CATEGORIAS)],
            }
        )
    return ventas
```

Genere un conjunto de datos con 100,000 ventas:

```
ventas = generar_ventas(n=100_000, seed=1234)
```

2. Implementación de operaciones

Realice dos operaciones sobre este conjunto de datos, cada una en dos variantes.

a. Calcular precios con IVA (21%):
   - **Versión A.** Use una *list comprehension* para crear una nueva lista llamada `precios_con_iva_1` que contenga todos los precios con el 21 % de IVA ya calculado.
   - **Versión B.** Use una expresión generadora para crear un objeto `precios_con_iva_2` que represente la operación, pero sin calcular ni almacenar los resultados todavía.

b. Filtrar ventas de “electrónica”:

- **Versión A.** Use una *list comprehension* para crear una nueva lista llamada `electronica_1` que contenga todas las ventas de la categoría "electrónica".
- **Versión B.** Use una expresión generadora para crear un objeto `electronica_2` que represente el filtro, sin materializar la lista.

3. Análisis y comparación

c. Calcule el total de las ventas de electrónica usando `sum()` en ambas variantes (`electronica_1` y `electronica_2`).
d. Para cada operación, antes y después de aplicar `sum()`, mida el tiempo de ejecución y el uso de memoria del objeto. Para el tiempo, podría usar `time.time()` del módulo `time`; para la memoria, `sys.getsizeof()` del módulo `sys`. Registre los resultados (si no observa diferencias claras, duplique `n` al generar el *dataset*).

4. Reutilización del objeto

e. Intente calcular el total de ventas de electrónica una segunda vez para ambos objetos (`electronica_1` y `electronica_2`).
f. Observe qué sucede en cada caso. ¿Qué objeto puede volver a usarse y cuál no?

5. Preguntas para reflexión

g. ¿Qué diferencias fundamentales encontró entre las dos maneras de procesar los datos? Considere cuándo se realiza la operación, el uso de memoria y el tiempo de ejecución.
h. ¿Qué ocurrió cuando intentó recorrer o usar el mismo resultado dos veces?
i. ¿Se comportaron de la misma manera la lista generada y el objeto generador? ¿Por qué cree que sucede esto?
j. ¿En qué situaciones elegiría un enfoque u otro? Dé un ejemplo de un escenario donde la ejecución A sea mejor y otro donde la B resulte más eficiente.